

**How Model Choice and Prompting Strategy Shape
Vulnerability Detection and Diagnostic Reliability in Local
LLMs**

Prepared by:	Muhammad Adil
Supervised by:	Independent

Abstract:

This study evaluates whether LLM choice and prompting strategy affect the ability of locally-run, open-weight models to detect and correctly classify security vulnerabilities in code. Three models of differing scale and specialization i.e. `phi3:mini`, `llama3.1:8b`, and `codellama:7b` were run via Ollama against a hand-curated, CWE-labeled dataset of 24 Python snippets across six vulnerability classes, under two prompting conditions (neutral vs. structured OWASP-guided), with two repetitions per configuration (288 total calls). All three models detected the presence of vulnerabilities more reliably than they identified the correct vulnerability class. `llama3.1:8b` and `codellama:7b` achieved high recall (0.92–1.00) but weak precision (~0.50–0.55), reflecting a strong bias toward over-flagging safe code. `phi3:mini` showed a more balanced precision/recall profile (0.65/0.625) but recorded 0% CWE-match accuracy, driven by outright fabrication of CWE identifiers rather than formatting mismatches. Prompting strategy had a small, inconsistent effect across models. These results indicate that binary detection and correct diagnostic labeling are separable capabilities, and that none of the three models would be safe to deploy as an unsupervised code-review aid.

1. Research Questions

- Does the choice of LLM affect vulnerability-detection accuracy?
- Does prompting strategy (a plain instruction vs. a structured, OWASP-guided instruction) improve detection?
- Which vulnerability categories (CWEs) are LLMs good or bad at catching?

2. Methodology

2.1. Local Model Setup

All inference was performed locally through `Ollama`, eliminating API costs, rate limits, and network dependency. Three models were selected to provide a meaningful contrast in scale and specialization rather than three models of comparable strength:

Model	Size	Role in comparison
phi3:mini (3.8B)	~2.3 GB	Small general-purpose model, "weaker" baseline
llama3.1:8b	~4.7 GB	Mid-size general-purpose model, "capable" condition
codellama:7b	~4 GB	Code-specialized model, tests whether coding skill transfers to security review

Models were pulled at their default Ollama quantization (Q4) and used without further fine-tuning. Inference ran on CPU with 16 GB RAM available; observed response latency ranged from roughly 8 to 90 seconds per call depending on model and prompt condition.

2.2. Dataset Construction

A hand-curated set of 24 short, self-contained Python snippets was built rather than drawn from an existing benchmark corpus, prioritizing clean CWE coverage and a balanced vulnerable/safe split over raw dataset size. Snippets were organized as matched pairs, a vulnerable ("bad") and a patched ("good") version of the same underlying task, across six CWE categories:

CWE	Category
CWE-89	SQL Injection
CWE-79	Cross-Site Scripting (XSS)
CWE-22	Path Traversal
CWE-798	Hardcoded Credentials
CWE-502	Insecure Deserialization
CWE-611	XML External Entity (XXE) Injection

Each category contributed four snippets (two vulnerable, two safe), for 24 snippets total, evenly split between vulnerable (12) and safe (12) examples. Ground truth (snippet_id, filepath, cwe, is_vulnerable, category_name) was recorded in a CSV file that served as the sole scoring reference throughout the study.

2.3. Prompt Conditions

Two prompt conditions were tested, both constrained to a fixed output format (VULNERABLE: yes/no, CWE: [...], EXPLANATION: [...]) to allow reliable automated parsing in the absence of a structured-output API:

- **Neutral condition:** a plain instruction to review the code and determine whether it contains a security vulnerability.
- **OWASP-guided condition:** an instruction framing the model as a security code reviewer and explicitly naming common vulnerability classes (injection flaws, broken access control, insecure deserialization, hardcoded secrets, improper input handling) in line with OWASP Top 10 guidance.

A brief pilot run on three snippets confirmed that all three models reliably followed the requested output format before the full batch was executed.

2.4. Evaluation Pipeline

Each of the 24 snippets was submitted to all three models under both prompt conditions, with two repetitions per configuration at `temperature=0.3` ($24 \times 3 \times 2 \times 2 = 288$ calls). Responses were parsed via regular expression into `predicted_vulnerable` (boolean), `predicted_cwe`, and a free-text explanation. Results were checkpointed to disk after every call to protect against interruption during the multi-hour run.

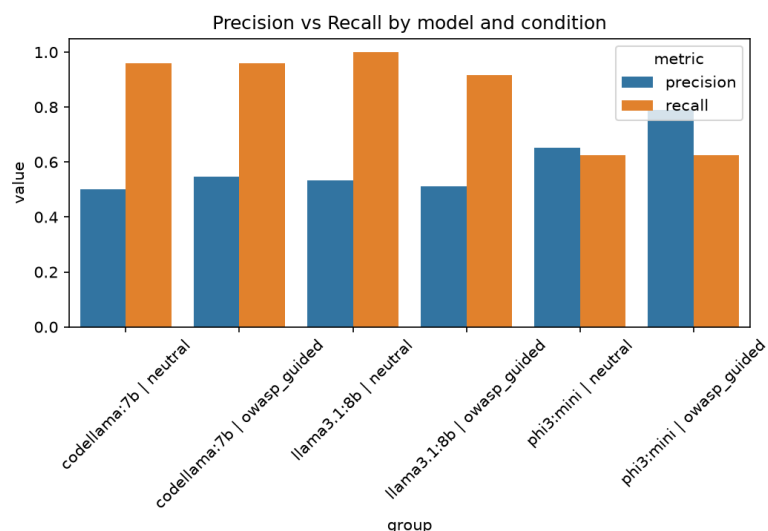
2.5. Scoring

Binary detection was scored against `is_vulnerable` using standard classification metrics i.e. accuracy, precision, recall, and F1, computed per model $\times$ condition. A stricter secondary metric, CWE-match accuracy, was computed on the subset of snippets that were truly vulnerable, asking not just "did the model flag this as vulnerable" but "did it also name the correct CWE." The parse failure rate was tracked but was 0% across all 288 responses, indicating all three models adhered to the requested format consistently.

3. Results

3.1. Overall Detection Performance by Model

Model	Condition	n	Accuracy	Precision	Recall	F1
codellama:7b	neutral	48	0.500	0.500	0.958	0.657
codellama:7b	owasp_guided	48	0.583	0.548	0.958	0.697
llama3.1:8b	neutral	48	0.563	0.533	1.000	0.696
llama3.1:8b	owasp_guided	48	0.521	0.512	0.917	0.657
phi3:mini	neutral	48	0.646	0.652	0.625	0.638
phi3:mini	owasp_guided	48	0.729	0.789	0.625	0.698



`codellama:7b` and `llama3.1:8b` both exhibit near-ceiling recall (0.92 – 1.00), meaning they rarely fail to flag a genuinely vulnerable snippet. This comes at a substantial cost to precision (~0.50 – 0.55), roughly half of the snippets these two models labeled "vulnerable" were in fact safe. A representative case observed during the pilot run illustrates this failure mode directly `sqli_good_1.py`, a parameterized (safe) query, was flagged as vulnerable with a fabricated justification describing the query placeholder as "unescaped."

`phi3:mini` shows the inverse profile: more conservative in calling code vulnerable, yielding better precision (0.65–0.79) but weaker recall (0.625 in both conditions), meaning it misses a larger share of genuinely vulnerable snippets than the other two models.

3.2. Effect of Prompting Strategy

The OWASP-guided prompt produced a real but inconsistent effect across models, undercutting any simple "structured prompting improves detection" narrative:

- For `phi3:mini`, the guided prompt meaningfully improved precision (0.652 → 0.789) with no change to recall.
- For `codellama:7b`, the effect was marginal (F1 0.657 → 0.697).
- For `llama3.1:8b`, the guided prompt slightly *degraded* performance, reducing an already precision-poor, recall-heavy profile further (F1 0.696 → 0.657).

Prompting strategy therefore interacts with the underlying model rather than uniformly improving detection quality.

3.3. Per-CWE / Diagnostic Accuracy

Model	Condition	CWE-match accuracy	n (vulnerable snippets)
codellama:7b	neutral	0.333	24
codellama:7b	owasp_guided	0.250	24
llama3.1:8b	neutral	0.417	24
llama3.1:8b	owasp_guided	0.458	24
phi3:mini	neutral	0.000	24
phi3:mini	owasp_guided	0.000	24

llama3.1:8b and codellama:7b correctly named the CWE for 25 to 46% of the vulnerabilities they detected, modest, but non-trivial and well above chance for an open-ended numeric identifier.

phi3:mini scored exactly 0.0 in both conditions. Because a value of zero

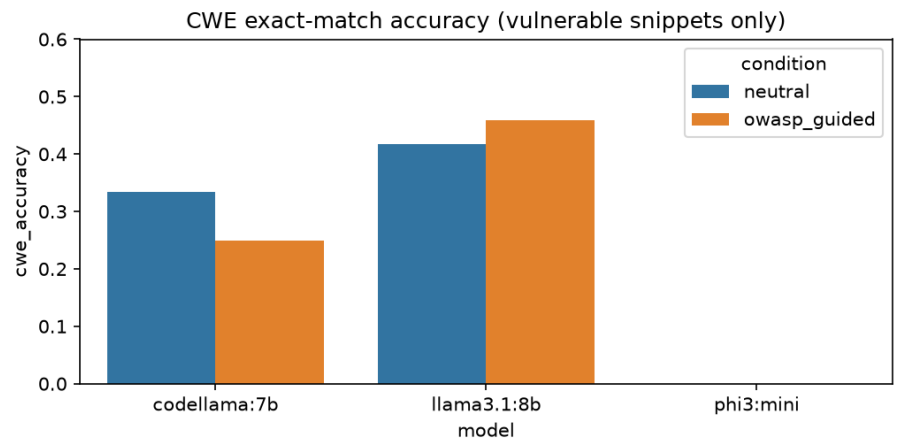
across 24 snippets and two conditions is unusual enough to warrant scrutiny before being reported as a genuine finding, the predicted CWE values were inspected directly rather than accepted at face value. The inspection ruled out a formatting artifact (e.g., "CWE89" vs. "CWE-89"): predicted values included identifiers such as CWE-257, CWE-23, CWE-046, CWE-006, CWE-07, and CWE-510; several of which do not correspond to any entry in the CWE taxonomy, and none of which are close numeric variants of the correct label (for example, the true label CWE-502 was predicted as CWE-257, a materially different identifier rather than a transcription slip). This pattern indicates outright fabrication rather than a normalization or scoring error: phi3:mini produces confident, plausible-looking CWE citations that do not correspond to a correct or even valid answer.

This reframes the model's earlier precision advantage. phi3:mini is not making *fewer* mistakes than the other two models, it is making a *different kind* of mistake. It is more conservative about calling code vulnerable at all, but when it does engage with the underlying reasoning, it fabricates supporting detail rather than declining to answer. For a real-world review workflow, this is arguably a more concerning failure mode than over-flagging, since a fabricated citation is more likely to mislead a reviewer who does not independently verify it.

A secondary observation noted here but not formally quantified, several of phi3:mini's NONE/missing CWE predictions occurred on snippets where its own vulnerable/not-vulnerable verdict was inconsistent across the two repetitions of the same snippet (e.g., s005, s006, s009, s017, s018, s021, s022). This suggests a within-model reliability question, the same input producing a different verdict across repeated runs, that falls outside this study's scope but is a natural extension for future work.

3.4. Response Time / Practicality

Observed per-call latency ranged from approximately 8 seconds (phi3:mini, best case) to roughly 90 seconds (codellama:7b, neutral condition, worst case observed



during the pilot), with typical calls in the 10–55 second range on the CPU-only 16 GB configuration used for this study. phi3:mini was consistently the fastest of the three models, consistent with its smaller parameter count. This has direct practical relevance, a tool intended for use in an interactive code-review workflow would need to weigh phi3:mini's speed and precision advantage against its complete failure to produce a trustworthy CWE label.

4. Discussion

Detection and diagnosis are separable capabilities. Across all three models, the ability to flag that *something* is wrong with a snippet does not imply the ability to say *what* is wrong. codellama:7b and llama3.1:8b detect broadly (high recall) but cannot reliably distinguish vulnerable from safe code (precision near 0.5, effectively closer to a coin flip once conditioned on a positive call), and even when they correctly flag a snippet, they name the correct CWE only 25 – 46% of the time. phi3:mini is more discriminating about *whether* to flag something, but that discrimination does not translate into accurate diagnosis in the slightest, its CWE accuracy is 0%, and the underlying cause is fabrication rather than caution.

Does code specialization help with security review specifically? codellama:7b, despite being trained with a coding focus, does not outperform the general-purpose llama3.1:8b on this task, the two show broadly similar precision/recall/F1 profiles, and codellama:7b actually posts the lowest CWE-match accuracy of the two under the OWASP-guided condition (0.250 vs. 0.458). This suggests that code-generation competence and security-vulnerability reasoning are, at least in this sample, separate skills rather than one implying the other.

Where do false negatives and false positives cluster? The dominant error pattern for codellama:7b and llama3.1:8b is false positives, over-flagging safe, patched code as vulnerable, as seen directly in the `sqli_good_1.py` misclassification. For phi3:mini, the dominant error pattern shifts toward false negatives on the binary call and outright fabrication on the diagnostic call. Neither failure mode is preferable in an unsupervised setting: over-flagging erodes reviewer trust and wastes triage time, while confident fabrication risks actively misleading a reviewer who does not double-check the citation.

Practical implication. None of the three models tested would be appropriate as an unsupervised vulnerability-detection tool in their current, default-quantized, zero-shot configuration. At best, they could serve as a first-pass triage signal for a human reviewer who treats both the vulnerability call and the CWE label as suggestions requiring verification, not conclusions.

5. Limitations

- Small snippet set ($n = 24$). Results should be read as directional and exploratory rather than statistically confirmed. No formal significance test was applied between model or condition pairs, this is flagged explicitly rather than over-claiming from a small sample.
- Local, quantized models. All models were run at Ollama's default (typically Q4) quantization. Full-precision or hosted-API versions of the same models may perform differently, and this affects how far these conclusions generalize.
- Single language focus. All snippets were written in Python, findings may not transfer to other languages.
- Self-curated ground truth. The dataset was not independently reviewed by a second rater, so there is no formal inter-rater reliability check on the ground-truth labels themselves, though the CWE classifications used are standard and unambiguous for the snippet patterns chosen.
- Reliability across repetitions was observed but not formally quantified. The inconsistency in phi3:mini's verdicts across repeated runs on identical snippets (Section 3.3) is a legitimate finding that would benefit from a dedicated consistency metric in future work.

6. Conclusion

Across three locally-run open-weight LLMs, no single model or prompting strategy produced trustworthy, unsupervised vulnerability detection. Model choice mattered less for binary detection quality, all three models landed in a similar, mediocre F1 range (0.64–0.70) than for the *type* of error each model makes, codellama:7b and llama3.1:8b over-flag safe code, while phi3:mini under-flags but fabricates diagnostic detail when it engages. Prompting strategy (neutral vs. OWASP-guided) had a real but small and model-dependent effect, improving one model's precision meaningfully while slightly harming another's. The clearest and most actionable finding is that CWE-level diagnostic accuracy lags binary detection accuracy substantially across the board, and in phi3:mini's case collapses to outright hallucination, underscoring that a model's stated vulnerability class should not be trusted without independent verification, regardless of how confident or well-formatted its explanation appears.

7. References

- CWE database, MITRE Corporation (<https://cwe.mitre.org>)
- OWASP Top 10 (2021) (<https://owasp.org/Top10/>)
- Ollama documentation (<https://ollama.com>)
- Model cards: phi3:mini, llama3.1:8b, codellama:7b
- Understanding the Effectiveness of Large Language Models in Detecting Security Vulnerabilities. <https://arxiv.org/pdf/2311.16169>
- Can You Really Trust Code Copilots? Evaluating Large Language Models from a Code Security. <https://arxiv.org/pdf/2505.10494>